

Frontend Lessons Learned

A running log of real bugs I hit and the underlying concept behind each one. Read this before starting a new project.

CSS Layout & Positioning

1. Invisible spacing isn't always margin/padding — check line-height

Mistake: Assumed a gap between a heading and subtitle was margin/padding, even after zeroing both out.

Concept: Every element has a default line-height (~1.15–1.2× font-size) that adds invisible space above/below text, scaling with font-size. If spacing persists after `margin:0; padding:0;`, check line-height next — set it explicitly (e.g. `line-height: 1.1`) instead of guessing at negative margins.

2. box-shadow doesn't take up layout space

Mistake: A card's bottom shadow looked "cropped" — cut off in a hard straight line.

Concept: box-shadow is purely decorative and painted outside the box; it never affects layout/height. If a sibling sits flush against the shadowed element with no gap, the sibling's opaque background paints over the shadow. Fix: add `margin-bottom` (or spacing at the grid/column level). Also check for `overflow:hidden` on any ancestor — it clips shadows on any edge.

3. box-shadow always renders on all 4 sides — there's no true "bottom-only" via one declaration

Mistake: Tried to get a shadow that only appears under an element.

Concept: Use a large offset + negative spread (4th value) to shrink/pull the shadow in so top/sides fade to near-nothing: `box-shadow: 0 15px 20px -10px #color;`. For pixel-perfect bottom-only, use a blurred `::after` pseudo-element positioned under the card instead.

4. position: fixed / sticky silently breaks near overflow and transform

Mistake: `position: fixed` (and later sticky) stopped working with no console error.

Concept: If any ancestor has `transform`, `filter`, `perspective`, or `will-change`, that ancestor becomes the new containing block — `fixed` no longer anchors to the viewport. Setting `overflow` (even just `overflow-x:hidden`) on `html` or `body` can break `fixed/sticky` children across browsers — apply it to `body` only, never `html`. `sticky` also needs its parent to be taller than itself — inside a stretched flex/grid row, `sticky` must go on an inner wrapper, not the stretched column itself. `position: fixed` also needs an explicit width (and left), or it shrinks to fit its content.

5. Nested <a> tags are invalid and cause unpredictable behavior

Mistake: Put an `<a>` inside another `<a>` for button-in-nav-link styling.

Concept: The browser auto-closes the first anchor when it hits the second, producing a different DOM structure than what was written. Never nest interactive elements — use a single element with combined classes instead.

6. align-self flips meaning when flex-direction changes (mobile vs desktop)

Mistake: align-self-center correctly vertically-centered nav items on desktop, but horizontally centered/bunched them on mobile.

Concept: align-self operates on the cross-axis, which rotates 90° when flex-direction switches from row (desktop navbar) to column (Bootstrap's collapsed mobile navbar). Scope direction-dependent alignment inside @media queries rather than applying one class across all breakpoints.

7. Absolutely positioned elements need a positioned parent + padding for their sibling

Mistake: An icon with position:absolute floated to the wrong place; once fixed, it overlapped input text.

Concept: position:absolute anchors to the nearest ancestor with position: relative/absolute/fixed/sticky — without one, it escapes up the DOM tree. Two positioned siblings with no z-index stack by DOM order (later = on top), which can bury an icon under a sibling's opaque background.

JavaScript Fundamentals

8. Checkbox .value is a static string — use .checked for state

Mistake: Read checkboxInput.value expecting it to reflect checked/unchecked; it always returned the hardcoded value="..." attribute regardless of state.

Concept: For checkboxes, .value = fixed HTML attribute. .checked = the actual boolean state (true/false). Same logic applies when resetting a form — set .checked = false, not .value = "".

9. <select> elements need .value, not the element itself

Mistake: Stored the whole <select> DOM element in a data object instead of its .value.

Concept: Forgetting .value on any input reference stores the element object, not user input — obvious in console.log (shows as an expandable DOM node) but easy to miss when just skimming code.

10. File inputs can't be read or set synchronously

Mistake: (A) Built an image path directly from files[0].name and expected it to display. (B) Tried fileInput.value = someContact.image to pre-fill an edit form. (C) Ran code right after reader.readAsDataURL() expecting the result to already be ready.

Concept: (A) Selecting a file never uploads/copies it anywhere — files[0] is just an in-memory reference; a hardcoded path only resolves if that file already exists on disk. Use FileReader.readAsDataURL() to embed the image as base64 for instant preview. (B) .value on a file input is read-only for security — the only legal assignment is "" ; anything else throws InvalidStateError and halts the function. (C) FileReader is asynchronous — any logic depending on the result must live inside reader.onload, not immediately after the call.

11. Incomplete ternary / missing function braces cause silent syntax failures

Mistake: condition ? valueA : — no false branch. Also: an if block left unclosed inside a function.

Concept: A syntax error like a missing ternary branch breaks the entire script from loading. Always give a ternary both outcomes, and count braces carefully in nested if/function blocks.

12. Duplicate id attributes in a loop only ever affect the first match

Mistake: Used the same `id="contact-image"` inside a for loop generating multiple cards, then tried `getElementById` to target "the current one."

Concept: IDs must be unique. `getElementById` always returns the first match in the DOM, silently. Anything built inside a repeating loop needs a different targeting strategy: pass the loop index into `onclick`, or better, pass this (the exact clicked element) so no lookup is needed at all.

13. DOM manipulation inside a "pure" data function runs at the wrong time

Mistake: Called `document.getElementById(...).remove()` from inside a helper function (`getInitials()`) that runs before the card it's trying to control has even been rendered.

Concept: Keep data-computation functions pure (input to output, no side effects). Decide DOM structure at render time, using the data itself (e.g. a ternary based on `contact.image` being truthy), not by reaching into the DOM from an unrelated helper.

14. Typos in property names return undefined, not an error

Mistake: `contactsList.lenght` (typo for `.length`) — loop condition became `i < undefined`, always false, so the loop silently never ran.

Concept: Unlike calling a misspelled function (which throws), reading a misspelled property just returns undefined quietly. If a loop/condition mysteriously never executes, suspect a typo'd property name first.

15. Rendered UI state must be derived from data on every render, not toggled independently

Mistake: Clicking a favorite button updated the array and toggled a CSS class on the clicked button — but the render function always output the "inactive" class by default, with no check against `contact.favorite`. Result: style needed two clicks to visibly change, and refreshing wiped it entirely.

Concept: Never let visual/DOM state and underlying data state drift independently. Every re-render must compute classes from the current data (`contact.favorite ? "active-class" : ""`), so the UI is always a direct reflection of state.

16. `&&` vs `||` when a value could match one of several acceptable forms

Mistake: Chained multiple format checks (plain vs +20-prefixed phone number) with `&&`, requiring all of them to be true simultaneously.

Concept: "Is X or Y or Z true" needs `||`. But the more robust fix for "same value, different formats" is normalizing both sides to one canonical format first (e.g. strip +20/0020/20 prefixes down to a local 0... form), then compare with a single `===`.

17. Regex: always give an "empty is OK" escape hatch for optional fields

Mistake: A strict format regex (email/phone) rejected an intentionally empty, non-required field.

Concept: `value.trim() === "" || regex.test(value.trim())` — empty passes automatically, anything non-empty must still fully match the pattern. Only apply this to genuinely optional fields; required fields should stay strict.

18. Copy-pasted validation functions can leave stale variable references behind

Mistake: Copied a working `validName()` to build `validEmail()/validPhone()`, but the "field is empty" branch inside still referenced the old input variable.

Concept: After copy-pasting a function as a template, search the entire body for every occurrence of the old identifier — it's easy to update the obvious top-level variable and miss one reused a few lines down.

General Debugging Habits Worth Repeating

- When something "does nothing" or seems inconsistent, check timing first — is the code trying to touch an element before it exists (async, or before a render function has run)?
- `console.log(JSON.stringify(value))` reveals hidden whitespace/formatting differences a plain `console.log` can hide.
- A single corrupted entry in `localStorage` from an earlier bug can keep causing errors even after the code is fixed — clear it and re-test with fresh data before concluding a fix didn't work.
- Red-outline-everything trick (`* { outline: 1px solid red !important; }`) is a fast way to visually spot which element is overflowing/misplaced.
- Two nearly-identical code paths (e.g. normal render vs. search-filtered render) must be fixed in both places — a bug fixed in one copy but not the other reappears the moment the second path runs.